

LESSON 02: VECTORIZATION

So far, what we've learned hasn't been all that useful. Sure, we can instantiate matrices, but we're here to make MATLAB work like a horse. One of the fundamental principles behind *clean & efficient* code is the vectorization of operations. By vectorizing, our code appears more like mathematical expressions found in textbooks, introduces fewer opportunities for errors, and runs much faster as it can utilize hardware vector operations!¹

Vector Generation

Say we want to create a vector $\mathbf{x} = [i : i \in \mathbb{Z}_8]$. We could do it like so:

```
1 x = 0:7
_____
1 x =
2
3      0      1      2      3      4      5      6      7
```

We achieved the following result by utilizing the `:` operator. This operator is useful for generating vectors, which we can use either for calculations or indexing into arrays.

The operator also allows us to specify a step increment other than one:

```
1 y = 2:-2:-12
_____
1 y =
2
3      2      0     -2     -4     -6     -8     -10     -12
```

But what if we want to create a vector over some range with a fixed amount of points (i.e. four evenly spaced values for $t \in [0, 1]$)? We could instead utilize `linspace()`:

```
1 t = linspace(0, 1, 4)
_____
1 t =
2
3      0      0.3333      0.6667      1.0000
```

¹https://www.mathworks.com/help/matlab/matlab_prog/vectorization.html

Indexing

Now, to *ruin* your day. MATLAB starts indexing at one...

Let's reuse $\mathbf{x}$ from the previous section. We can access the third element of $\mathbf{x}$ like so:

```
1 x(3)
_____
1 ans =
2
3      2
```

We can also access multiple elements at once:

```
1 x([1, 3, 6])
_____
1 ans =
2
3      0      2      5
```

Or if we're so inclined, the penultimate element:

```
1 x(end - 1)
_____
1 ans =
2
3      6
```

Matrices logically follow by adding another index:

```
1 A = [
2      1, 5
3      6, 2
4      ];
5
6 A(2, 2)
_____
1 ans =
2
3      2
```

We can also index *linearly*:

```
1 A(4)
_____
1 ans =
2
3      2
```

But that's an adventure for another time ;)

Size & Shape

When dealing with N-dimensional arrays, it's important to know how to get basic information related to the size and shape. Lucky for us, MATLAB offers a few convenient functions just for this!

- `length()` — the length of the largest array dimension
- `size()` — the size of the array
- `ndims()` — the number of array dimensions
- `numel()` — the number of array elements

Reshaping Arrays

Say we wanted to create the following matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} \quad (1)$$

We can agree that it's nothing more than

$$\mathbf{a} = [1, 2, 3, 4, 5, 6, 7, 8, 9], \quad (2)$$

where each column of $\mathbf{A}$ is every three elements of $\mathbf{a}$.

We can achieve this pragmatically with `reshape()`:

```
1 A = reshape(1:9, [3, 3])
```

```
1 A =
```

```
2
3     1     4     7
4     2     5     8
5     3     6     9
```

What's important to note is that for a reshape to be possible, the resultant array must have the same number of elements. Given this, we can let MATLAB do the hard work of determining a dimension for us:

```
1 A = reshape(1:9, 3, [])
```

```
1 A =
```

```
2
3     1     4     7
4     2     5     8
5     3     6     9
```

We could also *flatten* a matrix by reshaping, but you'll rarely see a MATLAB programmer do it this way. Instead, most programmers will linearly index all the elements of the matrix:

```
1 a = A(1:end)
```

```
1 a =
```

```
2
3     1     2     3     4     5     6     7     8     9
```

Broadcasting

Probably the most trippy thing when you first start using MATLAB is the concept of **broadcasting**. In essence, when working with array operations, MATLAB will take the liberty of *implicitly* doing a `repmat()` to make the sizes of operations work out.

Take for example:

```

1 u = [1, 2];
2 v = [3; 4; 5];
3
4 u + v

```

```

1 ans =
2
3      4      5
4      5      6
5      6      7

```

Strange, but it's as if we had performed the following operation manually:

```

1 [u; u; u] + [v, v]

```

Although broadcasting is *very* convenient, it is always waiting for you to drop your guard to stab you in the back when you least expect it. Having been a victim of MATLAB's senseless violence, you can do nothing besides being cautious around broadcasting operations to avoid introducing some *nasty* bugs.²

Function Calls

Depending on which function you call, its behavior on matrices will be different. Let's reuse matrix **A** from the previous section:

```

1 exp(A)

```

```

1 ans =
2
3      1.0e+03 *
4
5      0.0027      0.0546      1.0966
6      0.0074      0.1484      2.9810
7      0.0201      0.4034      8.1031

```

As you can see, `exp()` returns the exponential evaluated on each element of an array. However, this is not always the case; take, for example:

²Worst of all, you might even introduce a broadcasting operation unintentionally; that's when the real head-banging can begin!

```
1 sum(A)
```

```
1 ans =
```

```
2
```

```
3      6      15      24
```

Notice that `sum()` only evaluated the sum of each column of $\mathbf{A}$. This example illustrates why it's important to read a function's documentation *before* you use it, as it may not behave as you'd expect!

Now, we turn to an interesting experiment where we sum all the elements of a matrix. Let's generate a "large" matrix with uniformly distributed pseudorandom integers:

```
1 rng(0xc1c3eddb); % sets a constant seed for reproducibility
```

```
2
```

```
3 B = randi(10, 10e3, 10e3);
```

Now, let's compare the performance of summing all the elements of matrix $\mathbf{B}$ using two methods:

```
1 tic
```

```
2 sum(B(1:end))
```

```
3 toc
```

```
1 ans =
```

```
2
```

```
3      549929576
```

```
4
```

```
5 Elapsed time is 1.052319 seconds.
```

```
1 tic
```

```
2 sum(B, 'all')
```

```
3 toc
```

```
1 ans =
```

```
2
```

```
3      549929576
```

```
4
```

```
5 Elapsed time is 0.104034 seconds.
```

Wow, I don't know about you, but flattening is *significantly* slower than just calling `sum()` correctly. The reason for this dramatic difference is that we need to make a flattened *copy* of $\mathbf{B}$ in memory before we can sum all the values.

The moral of the story is that a bit of reading can save you a lot of time!